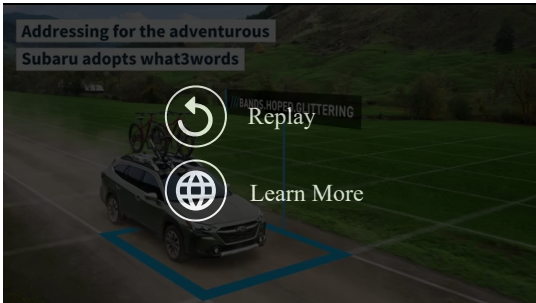




Subaru announces integration of what3words

Addressing for the adventurous - Subaru drivers can easily navigate to any precise 10ft x 10ft location in the world using just...

SPONSORED BY WHAT3WORDS

Learn More

JavaScript Maps

- A Map holds key-value pairs where the keys can be any datatype.
- A Map remembers the original insertion order of the keys.
- A Map has a property that represents the size of the map.

Map Methods

Method	Description
new Map()	Creates a new Map object
set()	Sets the value for a key in a Map
get()	Gets the value for a key in a Map
clear()	Removes all the elements from a Map
delete()	Removes a Map element specified by a key
has()	Returns true if a key exists in a Map
forEach()	Invokes a callback for each key/value pair in a Map
entries()	Returns an iterator object with the [key, value] pairs in a Map
keys()	Returns an iterator object with the keys in a Map
values()	Returns an iterator object of the values in a Map

Property	Description
size	Returns the number of Map elements

How to Create a Map

You can create a JavaScript Map by:

- Passing an Array to `new Map()`
- Create a Map and use `Map.set()`

new Map()

You can create a Map by passing an Array to the `new Map()` constructor:

Example

```
// Create a Map
const fruits = new Map([
  ["apples", 500],
  ["bananas", 300],
  ["oranges", 200]
]);
```

[Try it Yourself »](#)

Map.set()

You can add elements to a Map with the `set()` method:

Example

```
// Create a Map
const fruits = new Map();

// Set Map Values
fruits.set("apples", 500);
fruits.set("bananas", 300);
fruits.set("oranges", 200);
```

[Try it Yourself »](#)

The `set()` method can also be used to change existing Map values:

Example

```
fruits.set("apples", 500);
```

[Try it Yourself »](#)

Map.get()

The `get()` method gets the value of a key in a Map:

Example

```
fruits.get("apples"); // Returns 500
```

[Try it Yourself »](#)

ADVERTISEMENT

Map.size

The `size` property returns the number of elements in a Map:

Example

```
fruits.size;
```

[Try it Yourself »](#)

Map.delete()

The `delete()` method removes a Map element:

Example

```
fruits.delete("apples");
```

[Try it Yourself »](#)

Map.clear()

The `clear()` method removes all the elements from a Map:

Example

```
fruits.clear();
```

[Try it Yourself »](#)

Map.has()

The `has()` method returns true if a key exists in a Map:

Example

```
fruits.has("apples");
```

Try it Yourself »

Try This:

```
fruits.delete("apples");
fruits.has("apples");
```

Try it Yourself »

Maps are Objects

`typeof` returns object:

Example

```
// Returns object:
typeof fruits;
```

Try it Yourself »

`instanceof` Map returns true:

Example

```
// Returns true:
fruits instanceof Map;
```

Try it Yourself »

JavaScript Objects vs Maps

Differences between JavaScript Objects and Maps:

Object	Map
Not directly iterable	Directly iterable
Do not have a size property	Have a size property
Keys must be Strings (or Symbols)	Keys can be any datatype
Keys are not well ordered	Keys are ordered by insertion
Have default keys	Do not have default keys

Map.forEach()

The `forEach()` method invokes a callback for each key/value pair in a Map:

Example

```
// List all entries
let text = "";
fruits.forEach (function(value, key) {
  text += key + ' = ' + value;
})
```

[Try it Yourself »](#)

Map.entries()

The `entries()` method returns an iterator object with the [key,value] in a Map:

Example

```
// List all entries
let text = "";
for (const x of fruits.entries()) {
  text += x;
}
```

[Try it Yourself »](#)

Map.keys()

The `keys()` method returns an iterator object with the keys in a Map:

Example

```
// List all keys
let text = "";
for (const x of fruits.keys()) {
  text += x;
}
```

[Try it Yourself »](#)

Map.values()

The `values()` method returns an iterator object with the values in a Map:

Example

```
// List all values
let text = "";
for (const x of fruits.values()) {
  text += x;
}
```

[Try it Yourself »](#)

You can use the `values()` method to sum the values in a Map:

Example

```
// Sum all values
let total = 0;
for (const x of fruits.values()) {
  total += x;
}
```

[Try it Yourself »](#)

Objects as Keys

Being able to use objects as keys is an important Map feature.

Example

```
// Create Objects
const apples = {name: 'Apples'};
const bananas = {name: 'Bananas'};
const oranges = {name: 'Oranges'};

// Create a Map
const fruits = new Map();

// Add new Elements to the Map
fruits.set(apples, 500);
fruits.set(bananas, 300);
fruits.set(oranges, 200);
```

[Try it Yourself »](#)

Remember: The key is an object (apples), not a string ("apples"):

Example

```
fruits.get("apples"); // Returns undefined
```

[Try it Yourself »](#)